12: StatefulSets

In this chapter, you'll learn how to use *StatefulSets* to deploy and manage stateful applications on Kubernetes.

For the purposes of this chapter, we're defining a *stateful application* as one that creates and saves valuable data. An example might be an app that saves data about client sessions and uses it for future sessions. Other examples include databases and other data stores.

We'll divide the chapter as follows:

- StatefulSet theory
- Hands-on with StatefulSets

The theory section introduces you to the way StatefulSets work and what they bring to the game. But don't worry if you don't understand everything at first, you'll cover most of it again when you go through the hands-on section.

The theory of StatefulSets

It's often useful to compare StatefulSets with Deployments. Both are first-class API objects and follow the typical Kubernetes controller architecture. They're both implemented as controller that operate reconciliation loops watching the state of the cluster, via the API server, and moving the *observed state* into sync with *desired state*. Deployments and StatefulSets also support self-healing, scaling, updates, and more.

However, there are some vital differences between StatefulSets and Deployments. StatefulSets guarantee.

- Predictable and persistent Pod names
- Predictable and persistent DNS hostnames
- Predictable and persistent volume bindings

These three properties form the *state* of a Pod, sometimes referred to as its *sticky ID*. StatefulSets ensure this state/sticky ID is persisted across failures, scaling, and other scheduling operations, making them ideal for applications that require unique Pods that are not interchangeable.

As a quick example, failed Pods managed by a StatefulSet will be replaced by new Pods with the exact same Pod name, the exact same DNS hostname, and the exact same volumes. This is true even if the replacement Pod is started on a different cluster node. The same is not true of Pods managed by a Deployment.

The following YAML snippet shows *some* of the properties of a typical StatefulSet.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: tkb-sts
spec:
  selector:
    matchLabels:
      app: mongo
  serviceName: "tkb-sts"
  replicas: 3
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
      - name: ctr-mongo
        image: mongo:latest
        ...
```

The name of the StatefulSet is `tkb-sts` and it defines three Pod replicas running the `mongo:latest` image. You post this to the API server, it's persisted to the cluster store, the replicas are assigned to cluster nodes, and the StatefulSet controller monitors the state of the cluster making sure *observed state* matches *desired state*.

That's the big picture. Let's take a closer look at some of the major characteristics of StatefulSets before walking through an example.

StatefulSet Pod naming

All Pods managed by a StatefulSet get *predictable* and *persistent* names. These names are vital, and are at the core of how Pods are started, self-healed, scaled, deleted, attached to volumes, and more.

The format of StatefulSet Pod names is `<StatefulSetName>-<Integer>`. The integer is a *zero-based index ordinal*, which is just a fancy way of saying "number starting from 0". The first Pod created by a StatefulSet always gets index ordinal "0", and each subsequent Pod gets the next highest. Assuming the previous YAML snippet, the first Pod created will be called `tkb-sts-0`, the second will be called `tkb-sts-1`, and the third will be called `tkb-sts-2`.

Be aware that StatefulSet names need to be a valid DNS names, so no exotic characters.

Ordered creation and deletion

Another fundamental characteristic of StatefulSets is the controlled and ordered way they start and stop Pods.

StatefulSets create one Pod at a time, and always wait for previous Pods to be *running and ready* before creating the next. This is different from Deployments that use a ReplicaSet controller to start all Pods at the same time, causing potential race conditions.

As per the previous YAML snippet, `tkb-sts-0` will be started first and must be *running* and *ready* before the StatefulSet controller starts `tkb-sts-1`. The same applies to subsequent Pods – `tkb-sts-1` needs to be *running* and *ready* before `tkb-sts-2` starts etc. See Figure 12.1

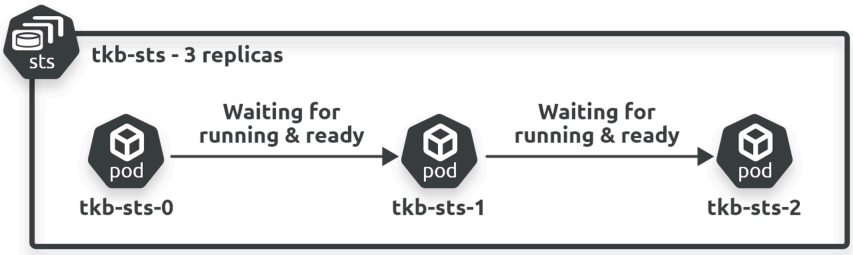

Figure 12.1

> **Note:** *Running* and *ready* are terms used to indicate all containers in a Pod are *running* and the Pod is *ready* to service requests.

Scaling operations are also governed by the same ordered startup rules. For example, scaling from 3 to 5 replicas will start a new Pod called `tkb-sts-3` and wait for it the be *running and ready* before creating `tkb-sts-4`. Scaling down follows the same rules in reverse – the controller terminates the Pod with the highest index ordinal (number) first, waits for it to fully terminate before terminating the Pod with the next highest ordinal.

Knowing the order in which Pods will be scaled down, as well as knowing that Pods will not be terminated in parallel, is a game-changer for many stateful apps. For example, clustered apps that store data can potentially lose data if multiple replicas go down at the same time. StatefulSets guarantee this will never happen. You can also inject other delays via things like `terminationGracePeriodSeconds` to further control the scaling down process. All in all, StatefulSets bring a lot to the table for clustered apps that create and store data.

Finally, it's worth noting that StatefulSet controllers do their own self-healing and scaling. This is architecturally different to Deployments which use a separate ReplicaSet controller for these operations.

Deleting StatefulSets

There are two major things to consider when deleting StatefulSets.

Firstly, deleting a StatefulSet does **not** terminate Pods in order. With this in mind, you may want to scale a StatefulSet to 0 replicas before deleting it.

You can also use `terminationGracePeriodSeconds` to further control the way Pods are terminated. It's common to set this to at least 10 seconds to give applications running in Pods a chance to flush local buffers and safely commit any writes that are still "in-flight".

StatefulSets and Volumes

Volumes are an important part of a StatefulSet Pod's *sticky ID* (state).

When a StatefulSet Pod is created, any volumes it needs are created at the same time and named in a special way that connects them to the right Pod . Figure 12.2 shows a StatefulSet called "ss" requesting 3 replicas. You can see how each Pod and volume (PVC) is created and how the names connect volumes to Pods.

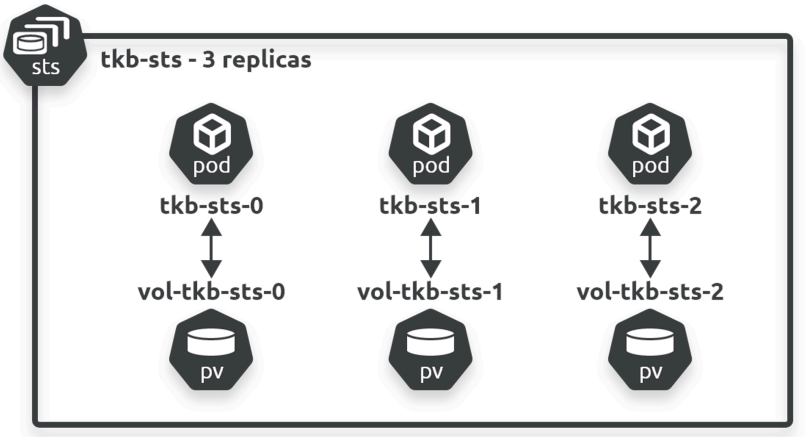

Figure 12.2

Volumes are appropriately decoupled from Pods via the normal Persistent Volume Claim system. This means volumes have separate lifecycles to Pods, allowing them to survive Pod failures and termination operations. For example, any time a StatefulSet Pod fails or is terminated, associated volumes are unaffected. This allows replacement Pods to attach to the same storage as the Pods they're replacing. This is true, even if replacement Pods are scheduled to different cluster nodes.

The same is true for scaling operations. If a StatefulSet Pod is deleted as part of a scale-down operation, subsequent scale-up operations will attach new Pods to the surviving volumes that match their names.

This behavior can be a life-saver if you accidentally delete a StatefulSet Pod, especially if it's the last replica!

Handling failures

The StatefulSet controller observes the state of the cluster and attempts to keep observed state in sync with desired state. The simplest example is a Pod failure. If you have a StatefulSet called `tkb-sts` with 5 replicas, and `tkb-sts-3` fails, the controller will start a replacement Pod with the same name and attach it to the same volumes.

However, if a failed Pod recovers after Kubernetes has replaced it, you'll have two identical Pods trying to write to the same volume. This can result in data corruption. As a result, the StatefulSet controller is extremely careful how it handles failures.

Possible node failures are very difficult to deal with. For example, if Kubernetes loses contact with a node, how does it know if the node is down and will never recover, or if it's a temporary glitch such as a network partition, a crashed kubelet, or the node is simply rebooting? To complicate matters further, the controller can't even force the Pod to terminate, as the local kubelet may never receive the instruction. With all of this in mind, manual intervention is needed before Kubernetes will replace Pods on failed nodes.

Network ID and headless Services

We've already said that StatefulSets are for applications that need Pods to be predictable and long-lived. As a result, other parts of the application as well as other applications may need to connect directly to individual Pods. To make this possible, StatefulSets use a ***headless Service*** to create predictable DNS hostnames for every Pod

replica. Other apps can then query DNS for the full list of Pod replicas and use these details to connect directly to Pods.

The following YAML snippet shows a headless Service called "mongo-prod" that is listed in the StatefulSet YAML as the *governing Service*.

```
apiVersion: v1
kind: Service
metadata:
  name: mongo-prod
spec:
  clusterIP: None
  selector:
    app: mongo
    env: prod
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: sts-mongo
spec:
  serviceName: mongo-prod       <<==== Governing Service
```

Let's explain the terms *headless Service* and *governing Service*.

A headless Service is just a regular Kubernetes Service object without an IP address (`spec.clusterIP` set to `None`). It becomes a StatefulSet's governing Service when you list it in the StatefulSet manifest under `spec.serviceName`.

When the two objects are combined like this, the Service will create DNS SRV records for each Pod replica that matches the label selector of the headless Service. Other Pods and apps can then find members of the StatefulSet by performing DNS lookups against the name of the headless Service. You'll see this in action later, and obviously applications need coding with this intelligence.

That covers most of the theory, let's walk through an example and see how it all comes together.

Hands-on with StatefulSets

In this section, you'll deploy a working StatefulSet. The example is intended to demonstrate the way StatefulSets work and reinforce what you've already learned. It's not intended as a production-grade application configuration.

The examples we'll show are on Google Kubernetes Engine (GKE). StatefulSets and what you're about to see work just fine on other Kubernetes clusters, but the StorageClass YAML file used in the examples is designed for GKE.

All of the YAML files we'll refer to are in the `statefulsets` folder of the book's GitHub repo. You can clone the repo with the following command.

```
$ git clone https://github.com/nigelpoulton/TheK8sBook.git
```

If you're following along, you'll deploy the following three objects:

1. A StorageClass
2. A headless Service
3. A StatefulSet

To make things easier to follow, you'll inspect and deploy each object individually. However, all three can be grouped in a single YAML file separated by three dashes (see `app.yml` in the `statefulsets` folder of the repo).

Deploying the StorageClass

StatefulSets that use volumes need to be able to create them dynamically. You need two objects to do this:

- StorageClass (SC)
- PersistentVolumeClaim (PVC)

The following YAML is from the `gcp-sc.yml` file and defines a StorageClass object called `flash` that will dynamically provision SSD volumes (`type=pd-ssd`) from the Google Cloud using the GKE persistent disk CSI driver (`pd.csi.storage.gke.io`). It will only work on Kubernetes clusters running on GCP or GKE with the CSI driver enabled.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: flash
provisioner: pd.csi.storage.gke.io
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
parameters:
  type: pd-ssd
```

Deploy the Storage class.

```
$ kubectl apply -f gcp-sc.yml
storageclass.storage.k8s.io/flash created
```

List your cluster's StorageClasses to make sure it was created correctly.

```
$ kubectl get sc
NAME   PROVISIONER            RECLAIMPOLICY  VOLUMEBINDINGMODE      ALLOWEXPANSION   AGE
flash  pd.csi.storage.gke.io  Delete         WaitForFirstConsumer   true             5s
```

With the StorageClass in place, Persistent Volume Claims (PVC) can use it to dynamically create new volumes. We'll circle back to this in a later step.

Creating a governing headless Service

When learning about headless Services, it can be useful to visualize a Service object with a head and a tail. The head is the stable IP address, and the tail is the list of Pods it sends traffic to. Therefore, a headless Service is a Service object without an IP address.

The sole purpose of a headless Service is to create DNS SRV records for Pods that match its label selector. Clients then need to know to use DNS to reach Pods instead of using the Service's ClusterIP. This is why a headless Service doesn't need a ClusterIP.

The following YAML is from the `statefulsets/headless-svc.yml` file and describes a headless Service called `dullahan` with no IP address (`spec.clusterIP: None`).

```
apiVersion: v1
kind: Service
metadata:
  name: dullahan
  labels:
    app: web
spec:
  ports:
  - port: 80
    name: web
  clusterIP: None
  selector:
    app: web
```

The only difference to a regular Service is that a headless Service must have its `clusterIP` set to `None`.

When combined with a StatefulSet, headless Services create predictable stable DNS entries for every Pod matching the StatefulSet's label selector. You'll see this in a later step.

Deploy the headless Service to your cluster.

```
$ kubectl apply -f headless-svc.yml
service/tkb-sts created
```

Verify the operation.

```
$ kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.0.0.1     <none>        443/TCP   145m
dullahan     ClusterIP   None         <none>        80/TCP    10s
```

Deploy the StatefulSet

With the StorageClass and headless Service in place, it's time to deploy the StatefulSet.

The following YAML is from the `sts.yml` file and defines the StatefulSet. Remember this is for learning purposes only, it's not intended as a production-grade deployment of an application.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: tkb-sts
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  serviceName: "dullahan"
  template:
    metadata:
      labels:
        app: web
    spec:
      terminationGracePeriodSeconds: 10
      containers:
      - name: ctr-web
        image: nginx:latest
        ports:
        - containerPort: 80
          name: web
        volumeMounts:
        - name: webroot
          mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
  - metadata:
      name: webroot
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: "flash"
      resources:
        requests:
          storage: 1Gi
```

There's a lot to take in, so let's step through the important parts.

The name of the StatefulSet is `tkb-sts`. This is important as it forms part of the name of every Pod the StatefulSet will create.

The `spec.replicas` field defines 3 replicas. These will be named `tkb-sts-0`, `tkb-sts-1`, and `tkb-sts-2`. They'll be created in numerical order, and the StatefulSet controller will wait for each replica to be running and ready before starting the next.

The `spec.serviceName` field designates the *governing Service*. This is the name of the headless Service created in the previous step and will create the DNS SRV records for each StatefulSet replica. It's called the *governing Service* because it's in charge of the DNS subdomain used by the StatefulSet.

The remainder of the `spec.template` section defines the Pod template that will be used to stamp out Pod replicas – things such as which container image to use and which ports to expose.

Last, but most certainly not least, is the `spec.volumeClaimTemplates` section.

Earlier in the chapter we said every StatefulSet that uses volumes needs to be able to create them dynamically. To do this you need a StorageClass and a PersistentVolumeClaim (PVC).

You've already created the StorageClass, so you're ready to go with that aspect. However, PVCs present an interesting challenge…

Each StatefulSet Pod needs its own unique storage. This means each one needs its own PVC. However, this isn't possible, as each Pod is created from the same template. Also, you'd have to pre-create a unique PVC for every potential StatefulSet Pod, which also isn't possible when you consider StatefulSets can be scaled up and down.

Clearly, a more intelligent StatefulSet-aware approach is needed. This is where *volume claim templates* come into play.

At a high-level, a volumeClaimTemplate dynamically creates a PVC each time the StatefulSet controller spawns a new Pod replica. It also contains the intelligence to name the PVC so it connects to the right Pod. This way, the StatefulSet manifest contains a Pod template section for stamping out Pod replicas, and a volume claim template section for stamping out PVCs.

The following YAML snippet shows the volumeClaimTemplate from the example. It defines a claim template called `webroot` requesting a 10GB volume from the `flash` StorageClass created earlier.

```
volumeClaimTemplates:
- metadata:
    name: webroot
  spec:
    accessModes: [ "ReadWriteOnce" ]
    storageClassName: "flash"
    resources:
      requests:
        storage: 10Gi
```

When the StatefulSet object is deployed, it will create three Pod replicas and three PVCs.

Deploy the StatefulSet and watch the Pods and PVCs get created.

```
$ kubectl apply -f sts.yml
statefulset.apps/tkb-sts created
```

Watch the StatefulSet ramp up to 3 running replicas. It'll take a minute or so for the 3 Pods and associated PVCs to be created – each Pod needs to be running and ready before the next one is started.

```
$ kubectl get sts --watch
NAME      READY   AGE
tkb-sts   0/3     10s
tkb-sts   1/3     23s
tkb-sts   2/3     46s
tkb-sts   3/3     69s
```

Notice how it took ∼23 seconds to start the first replica. Once that was running and ready, it took another 23 seconds to start the second, and then another 23 for the third.

Now check the PVCs.

```
$ kubectl get pvc
NAME                STATUS   VOLUME            CAPACITY   MODES   STORAGECLASS   AGE
webroot-tkb-sts-0   Bound    pvc-1146...f274   10Gi       RWO     flash          86s
webroot-tkb-sts-1   Bound    pvc-3026...6bcb   10Gi       RWO     flash          63s
webroot-tkb-sts-2   Bound    pvc-2ce7...e56d   10Gi       RWO     flash          40s
```

There are 3 new PVCs, each created at the same time as one of the StatefulSet Pod replicas. See how the name of each PVC is based on the name of the StatefulSet and the Pod it's associated with. The format of the PVC name is the volumeClaimTemplate name, followed by a dash, followed by the name of the Pod replica it's associated with.

```
Pod Name     |    PVC Name
tkb-sts-0   <->   webroot-tkb-sts-0
tkb-sts-0   <->   webroot-tkb-sts-1
tkb-sts-0   <->   webroot-tkb-sts-2
```

At this point, the StatefulSet is up and the app is running.

Testing peer discovery

You know that pairing a headless Service with a StatefulSet creates DNS SRV records for each Pod matching the Service's label selector. You already have a headless Service and 3 StatefulSet Pods running, so you should have three DNS SRV records – one for each Pod.

However, before testing this, it's worth taking a moment to understand how DNS hostnames and DNS subdomains work with StatefulSets.

By default, Kubernetes places all objects within the `cluster.local` DNS subdomain. You can choose something different, but most lab environments use this domain, so we'll assume it in this example. Within that domain, Kubernetes constructs DNS subdomains as follows:

```
<object-name>.<service-name>.<namespace>.svc.cluster.local
```

So far, you've got three Pods called `tkb-sts-0`, `tkb-sts-1`, and `tkb-sts-2` governed by the `dullahan` headless Service. This means the 3 Pods will have the following fully qualified DNS names:

- `tkb-sts-0.dullahan.default.svc.cluster.local`
- `tkb-sts-1.dullahan.default.svc.cluster.local`
- `tkb-sts-2.dullahan.default.svc.cluster.local`

To test this, you'll deploy a jump-pod that has the DNS `dig` utility pre-installed. You'll `exec` onto that Pod and use `dig` to query DNS for SRV records in the `dullahan.default.svc.cluster.local` subdomain.

Deploy the jump-pod from the `/StatefulSets/jump-pod.yml` file in the book's GitHub repo.

```
$ kubectl apply -f jump-pod.yml
pod/jump-pod created
```

Exec onto the Pod.

```
$ kubectl exec -it jump-pod -- bash
root@jump-pod:/#
```

Your terminal is now connected to the jump-pod. Run the following `dig` command from within the jump-pod.

```
$ dig SRV dullahan.default.svc.cluster.local
<Snip>
;; ADDITIONAL SECTION:
tkb-sts-0.dullahan.default.svc.cluster.local. 30 IN A 10.24.1.25
tkb-sts-2.dullahan.default.svc.cluster.local. 30 IN A 10.24.1.26
tkb-sts-1.dullahan.default.svc.cluster.local. 30 IN A 10.24.0.17
<Snip>
```

The query returns the fully qualified DNS names of each Pod, as well as each Pod's IP. Other applications, including the app itself, can use this method to discover an up-to-date list of Pods in the StatefulSet.

For this method of discovery to be useful, applications obviously need to know how to use it. For example, they need to know the name of the headless Service governing the StatefulSet and they need to know to query DNS.

Scaling StatefulSets

Each time a StatefulSet is scaled up, a Pod **and** a PVC is created. However, when scaling a StatefulSet down, the Pod is terminated but the PVC is not. This means future scale-up operations only need to create a new Pod, which is then connected to the surviving PVC. The StatefulSet controller includes all of the intelligence to track and manage all of this.

You currently have 3 StatefulSet Pod replicas and 3 PVCs. Edit the `sts.yml` file and change the replica count from 3 down to 2 and save your change. When you've done that, run the following command to re-post the YAML file to the cluster.

```
$ kubectl apply -f sts.yml
statefulset.apps/tkb-sts configured
```

Check the state of the StatefulSet and verify the Pod count has reduced to 2.

```
$ kubectl get sts tkb-sts
NAME      READY   AGE
tkb-sts   2/2     14m

$ kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
tkb-sts-0   1/1     Running   0          15m
tkb-sts-1   1/1     Running   0          15m
```

The number of Pod replicas has been successfully scaled down to 2, and the Pod with the highest index ordinal was deleted. However, you'll still have 3 PVCs – remember, scaling down and deleting Pod replicas does **not** delete associated PVCs.

Verify this.

```
$ kubectl get pvc
NAME                STATUS   VOLUME            CAPACITY   MODES   STORAGECLASS   AGE
webroot-tkb-sts-0   Bound    pvc-1146...f274   10Gi       RWO     flash          15m
webroot-tkb-sts-1   Bound    pvc-3026...6bcb   10Gi       RWO     flash          15m
webroot-tkb-sts-2   Bound    pvc-2ce7...e56d   10Gi       RWO     flash          15m
```

The fact that all three PVCs still exist means that scaling back up to three replicas only requires a new Pod to be created. As the name of the surviving PVC is `webroot-tkb-sts-2`, the StatefulSet controller knows to automatically connect it to the new Pod.

Edit the `sts.yml` file and increment the number of replicas back to 3 and save your change. When you've done that, re-post the YAML file to the API server with the following command.

```
$ kubectl apply -f sts.yml
statefulset.apps/tkb-sts configured
```

Give it a few seconds to deploy the new Pod and then verify with the following command.

```
$ kubectl get sts tkb-sts
NAME      READY   AGE
tkb-sts   3/3     20m
```

You have 3 Pods again. Describe the new `webroot-tkb-sts-2` PVC to verify it's mounted by the correct Pod.

```
$ kubectl describe pvc webroot-tkb-sts-2 | grep Mounted
Mounted By:    tkb-sts-2
```

It's worth noting that scale down operations will be put on hold if any of the Pods are in a failed state. This is to protect the resiliency of the app and integrity of any data.

Finally, it's possible to tweak the controlled and ordered starting and stopping of Pods via the StatefulSet's `spec.podManagementPolicy` property.

The default setting is `OrderedReady` and implements the strict methodical ordering previously explained. Setting the value to `Parallel` will cause the StatefulSet to act more like a Deployment where Pods are created and deleted in parallel. For example, scaling from 2 > 5 Pods will create all three new Pods instantaneously, and scaling down from 5 > 2 will delete three Pods in parallel. StatefulSet naming rules are still implemented, and the setting only applies to scaling operations and does not impact rolling updates.

Performing rolling updates

StatefulSets support rolling updates. You update the image version in the YAML and re-post it to the API server. Once authenticated and authorized, the controller replaces old Pods with new ones. However, the process always starts with the highest numbered Pod and works down through the set, one-at-a-time, until all Pods are on the new version. The controller also waits for each new Pod to be running and ready before replacing the one with the next lowest index ordinal.

For more information, run `$ kubectl explain sts.spec.updateStrategy`.

Test a Pod failure

A simple way to test a failure is to manually delete a Pod. This will delete the Pod but not the associated PVC. The StatefulSet controller will notice observed state vary from desired state, realise a Pod is missing, and start a new identical one in its place. This new Pod will have the same name and will be connected to the same PVC volume.

Let's test it.

Confirm you have three healthy Pods in your StatefulSet.

```
$ kubectl get pods
NAME        READY   STATUS   AGE
tkb-sts-0   1/1     Running  37m
tkb-sts-1   1/1     Running  37m
tkb-sts-2   1/1     Running  18m
```

You're about to delete the `tkb-sts-0` Pod. But before you do that, run a quick `$ kubectl describe` to confirm the PVC it's currently using. You don't *need* to do this, as you can deduce the name of the PVC from the name of the volumeClaimTemplate and the StatefulSet. However, it's good to confirm.

```
$ kubectl describe pod tkb-sts-0
Name:         tkb-sts-0
Namespace:    default
<Snip>
Status:       Running
IP:           10.24.1.13
<Snip>
Volumes:
  webroot:
    Type:       PersistentVolumeClaim (a reference to a PersistentVolumeClaim...)
    ClaimName:  webroot-tkb-sts-0
<Snip>
```

Based on the output (your lab will be different) the values are as follows:

- Name: `tkb-sts-0`
- PVC: `webroot-tkb-sts-0`

Let's delete the `tkb-sts-0` Pod and see if the StatefulSet controller recreates it.

```
$ kubectl delete pod tkb-sts-0
pod "tkb-sts-0" deleted

$ kubectl get pods --watch
NAME        READY   STATUS              RESTARTS   AGE
tkb-sts-1   1/1     Running             0          43m
tkb-sts-2   1/1     Running             0          24m
tkb-sts-0   0/1     Terminating         0          43m
tkb-sts-0   0/1     Pending             0          0s
tkb-sts-0   0/1     ContainerCreating   0          0s
tkb-sts-0   1/1     Running             0          34s
```

Placing a `--watch` on the `kubectl get` command shows the StatefulSet controller noticing the terminated Pod and creating a replacement – desired state is 3 replicas but observed state dropped to 2. As the failure is clean and easy to verify, the controller immediately kicked off the process to create a new Pod.

You can see the new Pod has the same name as the failed one, but does it have the same PVC?

The following command confirms it does.

```
$ kubectl describe pod tkb-sts-0 | grep ClaimName
    ClaimName:  webroot-tkb-sts-0
```

Recovering from potential node failures is a lot more complex and requires manual intervention. This is because failed nodes are notoriously hard to diagnose and confirm, and there's always a risk the failure could be transient. If the StatefulSet controller assumes a node has failed and replaces any StatefulSet Pods, but the node subsequently recovers, there's a chance of duplicate Pods on the network contending for the same storage. This can cause all kinds of bad things to happen, including data corruption.

Deleting StatefulSets

Earlier in the chapter, you learned that deleting a StatefulSet does not terminate managed Pods in order. Therefore, if your application is sensitive to ordered shutdown, you should scale the StatefulSet to 0 replicas before initiating the delete operation.

Scale your StatefulSet to 0 replicas and confirm the operation. It may take a few seconds for the set to scale all the way down to 0.

```
$ kubectl scale sts tkb-sts --replicas=0
statefulset.apps/tkb-sts scaled

$ kubectl get sts tkb-sts
NAME      READY   AGE
tkb-sts   0/0     86m
```

Once the StatefulSet has no replicas you can delete it.

```
$ kubectl delete sts tkb-sts
statefulset.apps "tkb-sts" deleted
```

You can also delete the StatefulSet by referencing its YAML file with `$ kubectl delete -f sts.yml`.

Feel free to exec onto the jump-pod and run another `dig` to prove the SRV records have been removed from DNS.

At this point, the StatefulSet object is deleted, but the headless Service, StorageClass, and jump-pod still exist. You may want to delete them as well.

Chapter Summary

In this chapter, you learned how StatefulSets create and manage applications that need to persist state.

They can self-heal, scale up and down, and perform updates. Rollbacks require manual attention.

Each Pod replica spawned by a StatefulSet gets a predictable and persistent name, DNS hostname, and unique set of volumes. These stay with the Pod for its entire lifecycle, including failures, restarts, scaling, and other scheduling operations. In fact, StatefulSet Pod names are integral to scaling operations and connecting to storage volumes.

However, StatefulSets are only a framework. Applications need to be written in ways to take advantage of the way StatefulSets behave.